Aeternum-core Smart Contract

Technical Documentation

Version: 2.0
Network: Ethereum
Contract: AeternumVault.sol
Prepared by: Ndubuisi Ugwuja — Founder, Aeternum Labs

Table of Contents

Overview

AeternumVault is a smart wallet vault built on Ethereum. It lets you store ETH safely, send and receive funds like a normal wallet, and set up an automatic backup plan in case you ever lose access to your wallet — or in case something happens to you.

Here is the core idea in plain terms:

You deposit ETH into your vault. You pick a backup address — a wallet owned by someone you trust, like a family member, or a second wallet you control. You set a timer, for example 365 days. If you never interact with your vault for that entire period, Chainlink Automation (a decentralized service that watches the blockchain) automatically sends your ETH to your backup address.

If you are still alive and active, you just ping the contract occasionally to reset the timer. No one can touch your funds except you. There are no admins, no custodians, and no backdoors.

AeternumVault v1 is completely free. There are no subscription fees or tiers. Every registered wallet receives identical protection — the only thing that differs between users is the inactivity period they choose.

How It Works

1. The Vault

When you register with AeternumVault, the contract creates a private vault in your name. Think of it like a safety deposit box with your name on it. Your ETH sits inside, and only you can access it under normal conditions.

You can use your vault like a normal wallet:

- Deposit ETH into it at any time
- Send ETH to any address directly from your vault
- Withdraw all your ETH back to your own wallet in one click
- Update your backup address or inactivity timer whenever you want
- Cancel your setup entirely and get all your ETH back

2. The Inactivity Timer

Every time you interact with your vault — deposit, send, withdraw, or even just call the ping function — the timer resets. The timer counts from your last interaction.

If your timer reaches zero (meaning you have been completely inactive for your chosen period), Chainlink Automation will detect this and send your ETH to your backup address automatically.

If you interact with your vault even once before the timer runs out, the timer resets and the process starts over.

3. The Ping Function

The ping function exists for one purpose: to prove you are still alive and active without moving any funds. Calling ping costs very little gas (around 6,000 gas units — a fraction of a cent) and resets your inactivity timer. If you go on a long trip, have a medical situation, or simply do not need to move funds for a while, just ping your vault to keep the timer fresh.

Key Actors

1. You (The User)

You control your vault entirely. Only you can deposit, send, withdraw, update your settings, or cancel your vault. No one else can access your funds.

2. Your Backup Address

This is the wallet that receives your ETH if you go inactive for too long. It can be:

- A wallet held by a trusted family member
- A hardware wallet you keep in cold storage
- A multisig wallet shared with people you trust
- Any Ethereum address that can receive ETH

Your backup address cannot access your vault while you are active. It only receives funds if the inactivity timer expires and automatic recovery runs.

3. Chainlink Automation

Chainlink Automation is a decentralized network of nodes that watches the blockchain for conditions you define. In AeternumVault, it watches all registered vaults and checks whether any have been inactive long enough for recovery to trigger.

Chainlink nodes call a function called checkUpkeep every 12 seconds. If any wallets are due for recovery, they call performUpkeep to execute the ETH transfer to the backup address. This is fully automated — no human being decides when to send your ETH.

4. The Chainlink Forwarder

When an upkeep is registered with Chainlink Automation, Chainlink deploys a unique forwarder contract specifically for that upkeep. This forwarder is the only address permitted to call performUpkeep on AeternumVault. Once the contract owner calls setForwarder() after deployment, every direct call to performUpkeep from any other address is permanently rejected. This prevents anyone from crafting and submitting fraudulent recovery data directly to the contract.

Automatic Recovery

1. How Recovery Is Triggered

Recovery happens automatically when all three of the following are true at the same time:

1. Your vault is registered and active
2. Your vault has a non-zero ETH balance
3. Your inactivity period has fully elapsed since your last interaction

When Chainlink detects this, it sends your entire ETH balance to your backup address in a single transaction.

2. What Happens If the Transfer Fails

Sometimes a backup address cannot receive ETH — for example, if it is a contract that was set up incorrectly. In this case, the recovery attempt fails and is counted as one failed attempt.

AeternumVault allows up to 3 failed recovery attempts. After the third consecutive failure, your vault is permanently marked as abandoned. This means:

- Your vault is removed from the automatic monitoring system
- Your ETH balance is still fully accessible to you — you can withdraw it or send it at any time
- You cannot re-register with the same backup address, but you can re-register with a new one

This 3-attempt limit exists to prevent Chainlink from running in an infinite loop trying to send ETH to a broken address, which would waste LINK tokens.

3. Carried Balance on Re-registration

If your vault was abandoned and you re-register without first withdrawing your balance, the contract automatically carries your existing balance into your new vault. You do not lose any funds by skipping the withdrawal step.

Architecture

1. Single Contract

AeternumVault is a single smart contract. There are no complex protocols, no liquidity pools, no lending, and no borrowing. Your ETH sits in the contract mapped to your address. It is never mixed with anyone else's funds.

Each vault is completely isolated. If something goes wrong with one user's setup, it has zero effect on any other user's vault.

2. The Registry

The contract keeps a list of all registered wallets that are currently active. This list is how Chainlink knows which wallets to monitor.

The registry uses an efficient data structure that allows wallets to be added and removed in constant time — meaning it takes the same amount of computing resources regardless of how many wallets are registered. This keeps gas costs manageable even when the protocol has hundreds of thousands of users.

3. How Scaling Works — The Rolling Cursor

This is the mechanism that allows a single Chainlink upkeep job to monitor hundreds of thousands of wallets indefinitely without needing to be split into multiple jobs.

The problem it solves: you cannot check all registered wallets in a single transaction. With 500,000 users, trying to scan every wallet at once would hit Ethereum's block gas limit and the transaction would fail.

AeternumVault uses a rolling cursor instead. Think of it as a spotlight that slowly sweeps across the entire registry, one window at a time.

Here is how it works in plain terms:

The contract keeps a pointer called the cursor. This pointer remembers where the last scan ended. Every time Chainlink runs, it scans a window of wallets starting from the cursor position — for example, wallets at positions 0 through 4,999.

If any wallets in that window are due for recovery, the cursor holds its position. The same window is re-scanned on the next call until all due wallets in that window have been processed and cleared.

Once the window is fully clear, and a minimum cooldown period has passed, the cursor advances to the next window — positions 5,000 through 9,999. This continues until the cursor reaches the end of the registry, at which point it wraps back to position 0 and starts over.

This design has two important properties. First, it is self-correcting — if the registry shrinks because users cancel their vaults and the cursor ends up pointing past the end of the list, it automatically resets to zero. Second, it scales permanently with one upkeep job. At 500,000 users, with a scan window of 5,000 and a one-hour cooldown between advances, the cursor completes a full sweep of all registered wallets roughly every 4 days. As long as there are always some wallets that are due for recovery, the cursor stays active and the upkeep job continues running.

4. Two Separate Size Limits

The contract separates the scanning step from the execution step, and each has its own size limit:

The scan limit (MAX_CHECK_UPKEEP_SIZE) controls how many wallets Chainlink reads during the checkUpkeep simulation. This runs entirely off-chain in Chainlink's environment and costs no gas, so it can be set high — 5,000 wallets per scan. The purpose is wide coverage: the more wallets scanned per tick, the faster the cursor covers the full registry.

The execution limit (MAX_PERFORM_UPKEEP_SIZE) controls how many recoveries are actually executed in a single on-chain transaction. This is capped at 50 because each recovery involves multiple on-chain state changes and an ETH transfer, and running too many in one transaction risks exceeding Ethereum's block gas limit. At 50 wallets per execution, the total gas cost sits comfortably within safe limits.

These two limits work together: Chainlink can find up to 5,000 due wallets in one scan but will only process 50 of them in the matching on-chain transaction. Any remaining due wallets in the same window are picked up on subsequent calls while the cursor holds its position.

Security Design

1. You Own Your Funds

AeternumVault is non-custodial. The protocol never owns your ETH — you do. The contract holds it on your behalf and executes your instructions. There is no admin key that can move your funds. There is no pause button that can freeze your vault. There is no upgrade mechanism that can change the rules after you have deposited.

What you see in the contract is what you get — permanently.

2. Reentrancy Protection

A reentrancy attack is a type of hack where a malicious contract tries to call back into a function before the first call has finished. AeternumVault uses two independent defences:

1. Checks-Effects-Interactions (CEI) pattern: The contract always updates all balances and state before sending any ETH. By the time any ETH transfer happens, your balance is already recorded as zero — so even if a malicious backup address tried to re-enter the contract, there is nothing left to steal.
2. ReentrancyGuard: Every function that sends ETH is protected by OpenZeppelin's ReentrancyGuard, a battle-tested lock that blocks re-entry at the code level.

3. Forwarder Access Control

Once the contract owner calls setForwarder() after deployment with the Chainlink-assigned forwarder address, performUpkeep becomes locked. Any call to performUpkeep from any address other than the registered Chainlink forwarder is immediately rejected. This prevents anyone from submitting crafted recovery data directly to the contract to manipulate which

wallets get processed. The forwarder address can only be set once and cannot be changed afterward.

4. Access Control

The contract uses one access control modifier:

- onlyRegistered: Only wallets that have registered a vault can call deposit, send, withdrawAll, ping, and configuration functions. An unregistered address calling these functions will be rejected.

There are no other privileged roles. No owner. No admin. No pause function.

5. Direct ETH Rejection

The contract's receive() function is set to revert. If you accidentally send ETH directly to the contract address without going through the deposit function, the transaction fails immediately and your ETH is returned.

Immutable Values and Limits

Setting	Value	What It Means
Minimum inactivity period	180 days (5 minutes for testnet)	The shortest timer a user is allowed to set
Maximum inactivity period	3,650 days	The longest timer a user is allowed to set — prevents funds from being permanently locked (10 years)
Maximum scan size	5,000 wallets	How many wallets Chainlink reads per checkUpkeep simulation (off-chain, no gas cost)
Maximum execution size	50 wallets	How many recoveries Chainlink can process per performUpkeep transaction (on-chain, gas-bound)
Maximum failed attempts	3	After 3 consecutive failed recovery transfers, the vault is permanently abandoned
Cursor advance interval	1 hour	Minimum cooldown between cursor advances when no wallets are due — controls how fast the cursor sweeps the full registry

---

Chainlink Automation Integration

1. How Chainlink Watches Your Vault

Chainlink Automation operates in two steps:

Step 1 — checkUpkeep (off-chain, free): Chainlink nodes simulate a call to checkUpkeep every 12 seconds. This function scans the current cursor window of registered wallets and checks whether any are due for recovery. This simulation costs nothing — it runs off-chain and does not spend any gas.

Step 2 — performUpkeep (on-chain, costs LINK): If checkUpkeep finds wallets due for recovery, Chainlink's forwarder contract submits a performUpkeep transaction on-chain. This is the real transaction that actually sends ETH to backup addresses. LINK is deducted from the upkeep balance to pay for this.

2. Stale Data Safety

There is a gap between when checkUpkeep identifies a wallet and when performUpkeep actually executes. In this gap, a user might have already pinged their vault or made a withdrawal, making recovery no longer needed. AeternumVault handles this safely: performUpkeep re-validates every wallet individually before sending any ETH. If a wallet is no longer due for recovery, it is silently skipped and no ETH is sent.

3. Setting Up Chainlink Automation

After deploying the contract:

1. Go to automation.chain.link and connect your wallet
2. Click Register new upkeep and select Custom Logic
3. Enter your deployed AeternumVault contract address
4. Leave checkData empty
5. Set the gas limit to 500,000
6. Fund the upkeep with LINK tokens from faucets.chain.link (Sepolia) or purchase LINK (mainnet)
7. After registration, open your upkeep details page and copy the Forwarder address
8. Call setForwarder(forwarderAddress) on your deployed contract — this permanently locks performUpkeep so only Chainlink's forwarder can call it. This step can only be performed once and cannot be reversed

One upkeep job is sufficient for any number of registered users. The rolling cursor architecture handles full registry coverage automatically over time.

Events

The contract emits the following events so that frontends, indexers, and monitoring tools can track everything that happens:

Event	When It Is Emitted
RecoveryRegistered	A user successfully registers a vault
Deposited	ETH is deposited into a vault
Sent	ETH is sent from a vault to another address
Withdrawn	A user withdraws all ETH from their vault
ActivityPinged	Any on-chain interaction resets the inactivity timer
BackupAddressUpdated	A user changes their backup address
InactivityPeriodUpdated	A user changes their inactivity period
RecoveryExecuted	Chainlink successfully sends ETH to a backup address
RecoveryFailed	A recovery attempt fails because the backup address cannot receive ETH
RecoveryAbandoned	A vault reaches 3 failed attempts and is permanently removed from monitoring
RecoveryCancelled	A user cancels their vault and withdraws their ETH
ForwarderSet	The Chainlink forwarder address is permanently registered on the contract

---

User Functions

1. register(backupAddress, inactivityPeriod)

Creates your vault. You provide your backup address and how long the inactivity timer should be in seconds. The minimum is 180 days and the maximum is 3,650 days (10 years). You can optionally deposit ETH at the same time by sending it with the transaction. If your wallet was previously abandoned and you still have a balance inside, that balance is automatically carried over into your new vault.

2. deposit()

Adds ETH to your existing vault. Send ETH with this transaction and it will be added to your vault balance. Your inactivity timer resets.

3. send(to, amount)

Sends ETH from your vault to any Ethereum address. Your vault balance decreases by the amount sent. Your inactivity timer resets.

4. withdrawAll()

Sends your entire vault balance back to your own wallet. Your inactivity timer resets. Your vault remains active — you can deposit again at any time.

5. ping()

Resets your inactivity timer without moving any funds. Costs minimal gas. Use this to prove you are active without making any transfers.

6. updateBackupAddress(newBackupAddress)

Changes your backup address to a new one. Your timer resets. The new address cannot be a backup that was previously abandoned due to failed transfers.

7. updateInactivityPeriod(newPeriod)

Changes how long your inactivity timer is. Must be between the minimum and maximum allowed values. Your timer resets.

8. cancelRecovery()

Withdraws all your ETH and removes your vault from the monitoring system in a single transaction. This is the clean exit function.

9. setForwarder(forwarderAddress)

Registers the Chainlink Automation forwarder address for this upkeep. Can only be called once. After this is set, only the registered forwarder may call performUpkeep. Get the forwarder address from the Chainlink Automation UI after registering your upkeep.

Read-Only Functions

1. isRegistered(wallet)

Returns true if the given wallet has an active vault registered.

2. getRecoveryConfig(wallet)

Returns the full configuration of a vault: backup address, inactivity period, current balance, last activity timestamp, active status, failed attempt count, and abandoned status.

3. isRecoveryDue(wallet)

Returns true if a vault is currently due for recovery — meaning the inactivity period has elapsed, the balance is non-zero, and the vault is active.

4. getTimeUntilRecovery(wallet)

Returns how many seconds remain before this vault becomes eligible for recovery. Returns 0 if recovery is already due.

5. getTotalRegistered()

Returns how many wallets are currently registered and active in the protocol.

6. getRegisteredWallets(start, end)

Returns a paginated slice of the registered wallets array.

7. getCheckCursor()

Returns the current position of the rolling cursor — which index in the registry the next scan window will begin from.

8. getLastCursorAdvance()

Returns the timestamp of the last time the cursor advanced during an idle window scan. Used to determine when the next cursor advance is eligible.

9. getForwarder()

Returns the registered Chainlink forwarder address. Returns the zero address if setForwarder has not been called yet.

10. isBackupAbandoned(backup)

Returns true if a backup address has been permanently blocked after failing to receive ETH across MAX_RECOVERY_ATTEMPTS consecutive recovery attempts. Any registration or backup update pointing to this address will be rejected.

Known Limitations

ETH only: The current version supports native ETH only. ERC-20 token support is planned for upcoming phase.

No yield: ETH sitting in your vault does not earn interest in the current version. Yield integration is planned for Phase 4.

EOA wallets only: You must interact with the contract using a standard Ethereum wallet. EIP-7702 hybrid account integration is planned for Phase 2.

Forwarder window: Between contract deployment and calling setForwarder(), performUpkeep is open to any caller. On Sepolia this is acceptable for testing. On mainnet, setForwarder() should be called immediately after deployment in the same deployment script to close this window.

External audit pending: AeternumVault has undergone comprehensive internal security testing including over 127 unit, fuzz, and invariant tests — covering 100% of contract functions and all identified branch conditions — plus static analysis with Slither and property-based fuzzing with Echidna across 50,000+ randomized call sequences. An independent external audit by a professional security firm is required before mainnet deployment.

Next Steps

Phase 2 — Aeternum Hybrid Wallet

Transforming Aeternum from a vault-based recovery system into a self-custodial hybrid wallet where users never deposit assets into a separate escrow contract. Assets stay in a wallet the user fully owns, while AeternumRecoveryManager monitors it and executes recovery automatically when inactivity conditions are met.

1. The Core Shift

In Phase 1, users deposit ETH into a vault contract for safekeeping. In Phase 2, the vault disappears. Users hold assets in their own Aeternum wallet — an address that looks and behaves exactly like a normal Ethereum wallet with a seed phrase — while the recovery layer operates silently in the background.

The wallet is built on EIP-7702, a standard that lets a regular EOA delegate execution to a smart contract without changing its address or requiring the user to deploy anything. The user gets one address, one backup phrase, and all the capabilities of a smart contract wallet — batched transactions, gas sponsorship, one-click DeFi interactions — without the friction of traditional smart contract wallets.

From the outside, the Aeternum wallet looks and feels like any other mobile wallet — e.g. MetaMask. Under the hood, it is programmable.

2. Multi-Asset Support

The Aeternum hybrid wallet supports ETH and ERC-20 tokens from launch. Each token balance is tracked independently per user in the RecoveryManager. When recovery triggers, the contract transfers all registered token balances alongside the ETH balance to the backup address in a single execution. The beneficiary receives everything the wallet held — not just the ETH.

Key considerations:

- If a single token transfer fails during batch recovery execution, that token's failure is recorded and retried on the next performUpkeep cycle, the remaining assets still transfer — one broken token cannot block recovery of everything else
- Token allowances granted to the RecoveryManager are revocable at any time. Revoking before recovery triggers causes that token to be skipped silently while all other assets recover normally

3. Activity Detection — No Ping Required

In Phase 1, users must call the ping function explicitly to prove liveness when they are not moving funds. In Phase 2, this requirement is removed entirely. Simply opening the Aeternum app resets the inactivity timer automatically. The app sends a lightweight signed interaction to the RecoveryManager in the background when the user opens it, recording the current timestamp as the last activity point.

This change has two consequences. First, it eliminates a step that most users would eventually forget. If your backup period is 365 days, you just need to open your wallet app once a year — the same behavior expected of any wallet user who occasionally checks their balance. Second, it makes the recovery guarantee more accurate: the timer reflects genuine wallet engagement rather than manual pings that a user might schedule artificially without actually being in good health or in control of their keys.

The ping function remains available on-chain for users who want to prove liveness from a separate interface or scripted environment, but it is no longer the primary mechanism.

4. How Recovery Works in Phase 2

The AeternumRecoveryManager monitors all registered Aeternum wallets using the same rolling cursor architecture from Phase 1. When a wallet's inactivity period expires, Chainlink Automation calls performUpkeep(), which calls _executeRecovery() on the wallet. ETH and all registered ERC-20 token balances move from the Aeternum wallet to the address the user designated as their backup — just like Phase 1.

Ownership rotation — where control of the smart account itself transfers to the backup rather than assets moving — is reserved as a future opt-in feature once Aeternum wallets are common enough as backup addresses to make it meaningful. Asset transfer remains the default recovery execution path.

5. What Carries Forward from Phase 1

The rolling cursor architecture, MAX_RECOVERY_ATTEMPTS abandonment handling, Chainlink forwarder access control, and the stale data safety pattern all carry forward unchanged. The recovery execution model is the same — only the source of assets changes from a vault escrow mapping to the user's self-custodial wallet.

Phase 3 — Multichain Expansion

Expanding the Aeternum hybrid wallet from a single-chain ETH product into a multichain recovery system. Users protect ETH and ERC-20 assets across any supported EVM-compatible network, with automated recovery maintained independently on each chain.

1. Multichain Deployment

The Aeternum hybrid wallet and AeternumRecoveryManager are deployed as independent instances on each supported EVM network. Each deployment is fully self-contained — registration, inactivity monitoring, and Chainlink Automation upkeeps are managed on-chain per network with no cross-chain message passing. Assets on each chain recover independently to the designated backup address on that same chain.

2. Supported networks (Phase 3 target)

- Base
- Arbitrum One
- Optimism
- BNB Chain
- Polygon,
- zkSync

3. Native asset per chain

- ETH on Ethereum, Base, Arbitrum, and Optimism
- BNB on BNB Chain
- POL on Polygon.

Users protecting assets across multiple chains register separately on each chain. The backup address is configured per deployment and can differ across chains.

Phase 4 — Financial Primitive

Expanding the Aeternum hybrid wallet from a recovery and custody tool into a full financial primitive. The inheritance philosophy of Phase 1 remains the baseline — users can always use Aeternum exactly as they did in Phase 1, holding assets in their wallet with a backup address and inactivity timer, nothing more. Everything built in Phase 4 is additive. No existing behavior changes. No assets are routed anywhere without the user explicitly enabling their smart account to do so.

1. Opt-In Yield

Users who want their assets working while they are alive optionally enable yield routing in their wallets. Aeternum acts as the programmable account layer that routes assets to established yield protocols and does intelligent things with the returns. Aave, Lido, Compound, Morpho, and other audited protocols generate the yield. Aeternum handles the routing, rebalancing, and — critically — the recovery of those positions when inactivity triggers.

Users who enable yield choose a risk preference:

- Conservative: assets routed to Lido liquid staking, earning Ethereum staking rewards in stETH
- Moderate: assets routed to Aave lending markets, earning variable lending yield in aWETH
- Optimised: assets allocated across multiple protocols based on current yield conditions, rebalanced automatically by Chainlink Automation

2. What Yield Routing Adds for Opted-In Users

The yield-bearing positions are held natively in the Aeternum wallet and are fully recoverable. When recovery triggers for an opted-in user, the position is unwound and the full value — principal plus all accumulated yield — transfers to the backup address.

3. Rebasing Token Recovery

Users who opt into yield and hold positions through Aave and Lido accumulate rebasing tokens — assets like aWETH and stETH whose balances grow over time as interest or staking rewards accrue. These require dedicated recovery execution paths because their transfer mechanics differ from standard ERC-20 tokens.

stETH (Lido staking)

Recovery triggers conversion of stETH to ETH via the Curve stETH/ETH liquidity pool, accepting minor slippage in exchange for immediate settlement, then transfers the ETH with all accumulated staking rewards intact directly to the backup address.

aWETH (Aave lending)

Recovery calls Aave Pool's withdraw() function to redeem the underlying WETH plus all accrued interest, unwraps WETH to ETH, and transfers the full amount to the backup address. If Aave pool utilization is temporarily at 100% and withdrawal is blocked, recovery retries on subsequent Chainlink Automation cycles using the existing MAX_RECOVERY_ATTEMPTS logic rather than failing immediately.

4. The Automated Disbursement Layer

This is where Aeternum becomes genuinely novel: A user holds 50 ETH in their Aeternum wallet. They choose to keep 30 ETH in the default recovery-only mode. They route 20 ETH to Aave for yield. They configure a disbursement rule: send 0.5 ETH to their children's wallets on the first of every month, and if they go inactive for 365 days, send everything to their designated backup address. Chainlink Automation executes all of it. The inheritance trigger and the disbursement triggers use the same underlying infrastructure — a monitored wallet with programmable execution rules.

This model extends into institutional use cases. A company treasurer can automate payroll disbursements from a yield-bearing treasury account. A DAO can automate grant distributions with built-in inheritance fallback. A fund manager can configure recurring allocations with automatic recovery if the manager becomes unreachable. The inheritance feature remains the foundation. Disbursement automation is the layer institutions build on top of it.

5. Enterprise APIs

Phase 4 exposes Aeternum's infrastructure as a programmable API surface for institutions building on top of it. Companies building custody tools, compliance systems, payroll infrastructure, or wallet products can access the recovery tools, disbursement engine, and yield routing layer programmatically.

Long-Term Direction

The four phases follow a single thread. Phase 1 validates the recovery mechanism at scale with a real user base and establishes trust. Phase 2 removes the vault and gives users a wallet they actually want to use every day — one that happens to have recovery built in. Phase 3 extends that wallet to every chain where users hold assets. Phase 4 makes those assets optionally productive, automates financial management for users who want it, and turns Aeternum into a platform that institutions and individuals build on.

The goal throughout is unchanged from the first line of Phase 1: if you ever lose access to your wallet or something happens to you, your assets go to the account you chose. Every feature built across these phases exists in service of that promise — making it more reliable, more comprehensive, and more accessible to anyone who needs it.

Frequently Asked Questions

Can the protocol take my ETH?

No. There is no admin key, no owner function, and no mechanism of any kind that allows anyone — including the people who built this — to access your vault balance. Your ETH can only go to you or your designated backup address.

Why is it free?

AeternumVault v1 has no subscription fees or tiers. The goal for this version is to make the service as accessible as possible and let users experience the product without any financial commitment upfront. Revenue will be introduced in future versions through streams that do not touch user deposits under any circumstances.

What happens if I forget to ping and my timer expires?

Your ETH is sent to your backup address automatically. If your backup address is a wallet you also control, you can simply move the ETH back. If you want to continue using the vault, register again.

What if I lose access to my backup address?

If you still control your primary wallet, this is not a problem. You can update your backup address at any time by calling updateBackupAddress(). The backup address only matters if recovery triggers after your inactivity period expires.

What if Chainlink goes down?

Chainlink is a decentralized network with thousands of nodes. A complete outage is extremely unlikely. If Chainlink were somehow unavailable, your ETH would remain in your vault until service resumes. The worst case is a delay in recovery — your funds are never at risk.

Can I have multiple vaults?

Currently, each Ethereum address can register one vault. Upgrades are planned for future versions.

Is the contract upgradeable?

No. Each deployed version of AeternumVault is immutable and cannot be modified after deployment. There are no proxy upgrades, admin-controlled logic changes, or governance mechanisms that can alter how an existing vault contract behaves once users deposit into it.

This is intentional. The rules you agree to when using a specific contract version remain fixed for the lifetime of that deployment. Aeternum Labs will continue developing new versions that are deployed as entirely new contracts. Users choose whether to remain on their current version or migrate to a newer one.

What is the forwarder and why does it matter?

The forwarder is a contract that Chainlink Automation deploys specifically for your upkeep registration. After calling setForwarder(), the AeternumVault contract will only accept performUpkeep calls from that forwarder address. This prevents anyone from directly submitting crafted recovery data to the contract. Without this protection, a sophisticated attacker could in theory call performUpkeep with a handcrafted list of wallet addresses and attempt to manipulate which recoveries get processed.
